

OOP PHP

Zdroj: https://www.w3schools.com/php/php_oop_what_is.asp

class

Fruit

Třída je tedy šablona pro objekty a objekt je instancí třídy.

Když jsou jednotlivé objekty vytvořeny, zdědí všechny vlastnosti a chování ze třídy, ale každý objekt bude mít různé hodnoty vlastností.

Definujte třídu

Třída je definována pomocí **class** klíčového slova, za kterým následuje název třídy a dvojice složených závorek ({}). Všechny jeho vlastnosti a metody jdou do rovnátek:

```
<?php
class Fruit {
    // code goes here...
}
?>
```

Níže deklarujeme třídu s názvem Fruit sestávající ze dvou vlastností (\$name a \$color) a dvou metod set_name() a get_name() pro nastavení a získání vlastnosti \$name:

```
<?php
class Fruit {
    // Properties
    public $name;
    public $color;

    // Methods
    function set_name($name) {
        $this->name = $name;
    }
    function get_name() {
        return $this->name;
    }
}
?>
```

Definujte objekty

Třídy nejsou nic bez předmětů! Z jedné třídy můžeme vytvořit více objektů. Každý objekt má všechny vlastnosti a metody definované ve třídě, ale budou mít různé hodnoty vlastností.

Objekty třídy se vytvářejí pomocí **new** klíčového slova.

V níže uvedeném příkladu jsou \$jablko a \$banán instancemi třídy Ovoce:

```
<?php
class Fruit {
    // Properties
    public $name;
    public $color;

    // Methods
    function set_name($name) {
        $this->name = $name;
    }
    function get_name() {
        return $this->name;
    }
}

$apple = new Fruit();
$banana = new Fruit();
$apple->set_name('Apple');
$banana->set_name('Banana');

echo $apple->get_name();
echo "<br>";
echo $banana->get_name();
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_class2

V níže uvedeném příkladu přidáme do třídy Fruit dvě další metody pro nastavení a získání vlastnosti \$color:

```
<?php
class Fruit {
    // Properties
```

```

public $name;
public $color;

// Methods
function set_name($name) {
    $this->name = $name;
}
function get_name() {
    return $this->name;
}
function set_color($color) {
    $this->color = $color;
}
function get_color() {
    return $this->color;
}
}

$apple = new Fruit();
$apple->set_name('Apple');
$apple->set_color('Red');
echo "Name: " . $apple->get_name();
echo "<br>";
echo "Color: " . $apple->get_color();
?>

```

Na tomto příkladu ukázat toString

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_class3

PHP – klíčové slovo \$this

Klíčové slovo \$this odkazuje na aktuální objekt a je dostupné pouze uvnitř metod.

Podívejte se na následující příklad:

```

<?php
class Fruit {
    public $name;
}

```

```
$apple = new Fruit();  
?>
```

Kde tedy můžeme změnit hodnotu vlastnosti \$name? Existují dva způsoby:

1. Uvnitř třídy (přidáním metody set_name() a použitím \$this):

```
<?php  
class Fruit {  
    public $name;  
    function set_name($name) {  
        $this->name = $name;  
    }  
}  
$apple = new Fruit();  
$apple->set_name("Apple");
```

```
echo $apple->name;  
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_this

Mimo třídu (přímoou změnou hodnoty vlastnosti):

```
<?php  
class Fruit {  
    public $name;  
}  
$apple = new Fruit();  
$apple->name = "Apple";
```

```
echo $apple->name;  
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_this2

PHP - instanceof

Klíčové slovo můžete použít **instanceof** ke kontrole, zda objekt patří do konkrétní třídy:

```
<?php
$apple = new Fruit();
var_dump($apple instanceof Fruit);
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_class4

!PHP – funkce __construct

Konstruktor umožňuje inicializovat vlastnosti objektu po vytvoření objektu.

Pokud vytvoříte `__construct()` funkci, PHP tuto funkci automaticky zavolá, když vytvoříte objekt z třídy.

Všimněte si, že funkce konstrukt začíná dvěma podtržítky (`__`)!

V níže uvedeném příkladu vidíme, že použití konstruktoru nám ušetří volání metody `set_name()`, která snižuje množství kódu:

```
<?php
class Fruit {
    public $name;
    public $color;

    function __construct($name) {
        $this->name = $name;
    }
    function get_name() {
        return $this->name;
    }
}
```

```
$apple = new Fruit("Apple");
echo $apple->get_name();
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_constructor

Další příklad:

Příklad

```
<?php
class Fruit {
    public $name;
    public $color;

    function __construct($name, $color) {
        $this->name = $name;
        $this->color = $color;
    }
    function get_name() {
        return $this->name;
    }
    function get_color() {
        return $this->color;
    }
}
```

```
$apple = new Fruit("Apple", "red");
echo $apple->get_name();
echo "<br>";
echo $apple->get_color();
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_constructor2

PHP - Funkce __destruct

Destruktor je volán, když je objekt zničen nebo je skript zastaven nebo ukončen.

Pokud vytvoříte `__destruct()` funkci, PHP tuto funkci na konci skriptu automaticky zavolá.

Všimněte si, že funkce destruct začíná dvěma podtržítky (`__`)!

Níže uvedený příklad obsahuje funkci `__construct()`, která se automaticky volá, když vytvoříte objekt ze třídy, a funkci `__destruct()`, která se automaticky volá na konci skriptu:

```
<?php
class Fruit {
```

```

public $name;
public $color;

function __construct($name) {
    $this->name = $name;
}
function __destruct() {
    echo "The fruit is {$this->name}.";
}
}

```

```

$apple = new Fruit("Apple");
?>

```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_destructor

```

<?php
class Fruit {
    public $name;
    public $color;

    function __construct($name, $color) {
        $this->name = $name;
        $this->color = $color;
    }
    function __destruct() {
        echo "The fruit is {$this->name} and the color is {$this->color}.";
    }
}

```

```

$apple = new Fruit("Apple", "red");
?>

```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_destructor2

!PHP - Modifikátory přístupu

Vlastnosti a metody mohou mít modifikátory přístupu, které řídí, kde k nim lze přistupovat.

Existují tři modifikátory přístupu:

- **public**- vlastnost nebo metoda jsou přístupné odkudkoli. Toto je výchozí nastavení
- **protected**- vlastnost nebo metoda jsou přístupné v rámci třídy a tříd odvozených z této třídy
- **private**- vlastnost nebo metoda je přístupná POUZE v rámci třídy

V následujícím příkladu jsme ke třem vlastnostem (název, barva a váha) přidali tři různé modifikátory přístupu. Zde, pokud se pokusíte nastavit vlastnost name, bude to fungovat dobře (protože vlastnost name je veřejná a lze k ní přistupovat odkudkoli). Pokud se však pokusíte nastavit vlastnost barvy nebo váhy, dojde k fatální chybě (protože vlastnosti barvy a váhy jsou chráněné a soukromé):

```
<?php
class Fruit {
    public $name;
    protected $color;
    private $weight;
}

$mango = new Fruit();
$mango->name = 'Mango'; // OK
$mango->color = 'Yellow'; // ERROR
$mango->weight = '300'; // ERROR
?>
```

V dalším příkladu jsme přidali modifikátory přístupu ke dvěma funkcím. Pokud se zde pokusíte zavolat funkci set_color() nebo set_weight(), dojde k závažné chybě (protože tyto dvě funkce jsou považovány za chráněné a soukromé), i když jsou všechny vlastnosti veřejné:

```
<?php
class Fruit {
    public $name;
    public $color;
    public $weight;

    function set_name($n) { // a public function (default)
        $this->name = $n;
    }
    protected function set_color($n) { // a protected function
        $this->color = $n;
    }
    private function set_weight($n) { // a private function
```



```

    $this->weight = $n;
}
}

$mango = new Fruit();
$mango->set_name('Mango'); // OK
$mango->set_color('Yellow'); // ERROR
$mango->set_weight('300'); // ERROR
?>

```

PHP – Co je dědičnost?

Dědičnost v OOP = Když třída pochází z jiné třídy.

Podřízená třída zdědí všechny veřejné a chráněné vlastnosti a metody od nadřazené třídy. Navíc může mít své vlastní vlastnosti a metody.

Zděděná třída je definována pomocí **extends** klíčového slova.

Podívejme se na příklad:

```

?php
class Fruit {
    public $name;
    public $color;
    public function __construct($name, $color) {
        $this->name = $name;
        $this->color = $color;
    }
    public function intro() {
        echo "The fruit is {$this->name} and the color is {$this->color}.";
    }
}

// Strawberry is inherited from Fruit
class Strawberry extends Fruit {
    public function message() {
        echo "Am I a fruit or a berry? ";
    }
}

$strawberry = new Strawberry("Strawberry", "red");
$strawberry->message();

```

```
$strawberry->intro();
```

```
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_inheritance

Příklad vysvětlen

Třída Jahoda je zděděna od třídy Fruit.

To znamená, že třída Strawberry může používat veřejné vlastnosti \$name a \$color a také veřejné metody __construct() a intro() ze třídy Fruit kvůli dědičnosti.

Třída Strawberry má také svou vlastní metodu: message().

PHP – dědičnost a modifikátor chráněného přístupu

V předchozí kapitole jsme se dozvěděli, že k **protected** vlastnostem nebo metodám lze přistupovat v rámci třídy a třídami odvozenými z této třídy. Co to znamená?

Podívejme se na příklad:

```
<?php
class Fruit {
    public $name;
    public $color;
    public function __construct($name, $color) {
        $this->name = $name;
        $this->color = $color;
    }
    protected function intro() {
        echo "The fruit is {$this->name} and the color is {$this->color}.";
    }
}

class Strawberry extends Fruit {
    public function message() {
        echo "Am I a fruit or a berry? ";
    }
}
```

```
// Try to call all three methods from outside class
$strawberry = new Strawberry("Strawberry", "red"); // OK. __construct() is public
$strawberry->message(); // OK. message() is public
$strawberry->intro(); // ERROR. intro() is protected
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_inheritance3

Ve výše uvedeném příkladu vidíme, že vše funguje dobře! Je to proto, že metodu (intro()) voláme **protected** zevnitř odvozené třídy.

PHP - Přepsání zděděných metod

Zděděné metody lze přepsat předefinováním metod (použijte stejný název) v podřazené třídě.

Podívejte se na příklad níže. Metody __construct() a intro() v podřazené třídě (Strawberry) přepíšou metody __construct() a intro() v nadřazené třídě (Fruit):

```
<?php
class Fruit {
    public $name;
    public $color;
    public function __construct($name, $color) {
        $this->name = $name;
        $this->color = $color;
    }
    public function intro() {
        echo "The fruit is {$this->name} and the color is {$this->color}.";
    }
}

class Strawberry extends Fruit {
    public $weight;
    public function __construct($name, $color, $weight) {
        $this->name = $name;
        $this->color = $color;
        $this->weight = $weight;
    }
}
```

```

}
public function intro() {
    echo "The fruit is {$this->name}, the color is {$this->color}, and the weight is
    {$this->weight} gram.";
}
}

```

```

$strawberry = new Strawberry("Strawberry", "red", 50);
$strawberry->intro();
?>

```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_inheritance4

PHP – konečné klíčové slovo

Klíčové **final** slovo lze použít k zabránění dědičnosti třídy nebo k zabránění přepsání metody.

Následující příklad ukazuje, jak zabránit dědění třídy:

```

<?php
final class Fruit {
    // some code
}

// will result in error
class Strawberry extends Fruit {
    // some code
}
?>

```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_inheritance5

Následující příklad ukazuje, jak zabránit přepsání metody:

Příklad

```

<?php
class Fruit {
    final public function intro() {
        // some code
    }
}

```

```
}  
}
```

```
class Strawberry extends Fruit {  
    // will result in error  
    public function intro() {  
        // some code  
    }  
}  
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_inheritance6

PHP - konstanty třídy

Jakmile je deklarována, konstanty nelze změnit.

Konstanty třídy mohou být užitečné, pokud potřebujete definovat nějaká konstantní data v rámci třídy.

Konstanta třídy je deklarována uvnitř třídy pomocí **const** klíčového slova.

Konstanty třídy rozlišují velká a malá písmena. Konstanty se však doporučuje pojmenovávat velkými písmeny.

Ke konstantě můžeme přistupovat z vnějšku třídy pomocí názvu třídy následovaného operátorem rozlišení rozsahu (**::**) následovaným názvem konstanty, jako zde:

Příklad

```
<?php  
class Goodbye {  
    const LEAVING_MESSAGE = "Thank you for visiting W3Schools.com!";  
}  
  
echo Goodbye::LEAVING_MESSAGE;  
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_class_constant

Nebo můžeme přistupovat ke konstantě zevnitř třídy pomocí **self** klíčového slova následovaného operátorem rozlišení rozsahu (**::**) následovaným názvem konstanty, jako zde:

Příklad

```
<?php
class Goodbye {
    const LEAVING_MESSAGE = "Thank you for visiting W3Schools.com!";
    public function byebye() {
        echo self::LEAVING_MESSAGE;
    }
}
```

```
$goodbye = new Goodbye();
$goodbye->byebye();
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_class_constant2

PHP – Co jsou abstraktní třídy a metody?

Abstraktní třídy a metody jsou, když má nadřazená třída pojmenovanou metodu, ale k plnění úkolů potřebuje její podřízené třídy.

Abstraktní třída je třída, která obsahuje alespoň jednu abstraktní metodu. Abstraktní metoda je metoda, která je deklarována, ale není implementována v kódu.

Abstraktní třída nebo metoda je definována pomocí **abstract** klíčového slova:

Syntax

```
<?php
abstract class ParentClass {
    abstract public function someMethod1();
    abstract public function someMethod2($name, $color);
    abstract public function someMethod3() : string;
}
?>
```

Při dědění z abstraktní třídy musí být metoda podřizené třídy definována se stejným názvem a se stejným nebo méně omezeným modifikátorem přístupu. Pokud je tedy abstraktní metoda definována jako chráněná, metoda podřizené třídy musí být definována jako chráněná nebo veřejná, ale ne soukromá. Také typ a počet požadovaných argumentů musí být stejný. Podřizené třídy však mohou mít navíc volitelné argumenty.

Když je tedy podřizená třída zděděna z abstraktní třídy, máme následující pravidla:

- Metoda podřizené třídy musí být definována se stejným názvem a znovu deklaruje nadřazenou abstraktní metodu
- Metoda podřizené třídy musí být definována se stejným nebo méně omezeným modifikátorem přístupu
- Počet požadovaných argumentů musí být stejný. Podřizená třída však může mít navíc volitelné argumenty

Podívejme se na příklad:

Příklad

```
<?php
// Parent class
abstract class Car {
    public $name;
    public function __construct($name) {
        $this->name = $name;
    }
    abstract public function intro() : string;
}

// Child classes
class Audi extends Car {
    public function intro() : string {
        return "Choose German quality! I'm an $this->name!";
    }
}

class Volvo extends Car {
    public function intro() : string {
        return "Proud to be Swedish! I'm a $this->name!";
    }
}
```

```

class Citroen extends Car {
    public function intro() : string {
        return "French extravagance! I'm a $this->name!";
    }
}

```

// Create objects from the child classes

```

$audi = new audi("Audi");
echo $audi->intro();
echo "<br>";

```

```

$volvo = new volvo("Volvo");
echo $volvo->intro();
echo "<br>";

```

```

$citroen = new citroen("Citroen");
echo $citroen->intro();
?>

```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_abstract

Příklad vysvětlen

Třídy Audi, Volvo a Citroen jsou zděděny z třídy Car. To znamená, že třídy Audi, Volvo a Citroen mohou používat veřejnou vlastnost \$name a také metodu public __construct() z třídy Car z důvodu dědičnosti.

Ale intro() je abstraktní metoda, která by měla být definována ve všech podřízených třídách a ty by měly vrátet řetězec.

PHP - Další příklady abstraktní třídy

Podívejme se na další příklad, kde má abstraktní metoda argument:

Příklad

```

<?php
abstract class ParentClass {
    // Abstract method with an argument
    abstract protected function prefixName($name);
}

```



```

class ChildClass extends ParentClass {
    public function prefixName($name) {
        if ($name == "John Doe") {
            $prefix = "Mr.";
        } elseif ($name == "Jane Doe") {
            $prefix = "Mrs.";
        } else {
            $prefix = "";
        }
        return "{$prefix} {$name}";
    }
}

```

```

$class = new ChildClass;
echo $class->prefixName("John Doe");
echo "<br>";
echo $class->prefixName("Jane Doe");
?>

```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_abstract2

Podívejme se na další příklad, kde abstraktní metoda má argument a podřízená třída má dva volitelné argumenty, které nejsou definovány v abstraktní metodě rodiče:

Příklad

```

<?php
abstract class ParentClass {
    // Abstract method with an argument
    abstract protected function prefixName($name);
}

class ChildClass extends ParentClass {
    // The child class may define optional arguments that are not in the parent's
    abstract method
    public function prefixName($name, $separator = ".", $greet = "Dear") {
        if ($name == "John Doe") {
            $prefix = "Mr";
        } elseif ($name == "Jane Doe") {
            $prefix = "Mrs";
        } else {

```

```

    $prefix = "";
}
return "{$greet} {$prefix}{$separator} {$name}";
}
}

```

```

$class = new ChildClass;
echo $class->prefixName("John Doe");
echo "<br>";
echo $class->prefixName("Jane Doe");
?>

```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_abstract3

PHP – co jsou rozhraní?

Rozhraní vám umožňují určit, jaké metody má třída implementovat.

Rozhraní usnadňují použití řady různých tříd stejným způsobem. Když jedna nebo více tříd používá stejné rozhraní, označuje se to jako "polymorfismus".

Rozhraní jsou deklarována pomocí **interface** klíčového slova:

Syntax

```

<?php
interface InterfaceName {
    public function someMethod1();
    public function someMethod2($name, $color);
    public function someMethod3() : string;
}
?>

```

PHP - rozhraní vs. abstraktní třídy

Rozhraní jsou podobná abstraktním třídám. Rozdíl mezi rozhraními a abstraktními třídami je:

- Rozhraní nemohou mít vlastnosti, zatímco abstraktní třídy mohou

- Všechny metody rozhraní musí být veřejné, zatímco metody abstraktní třídy jsou veřejné nebo chráněné
- Všechny metody v rozhraní jsou abstraktní, takže je nelze implementovat do kódu a klíčové slovo `abstract` není nutné
- Třídy mohou implementovat rozhraní a zároveň dědit z jiné třídy

PHP - Použití rozhraní

K implementaci rozhraní musí třída použít **implements** klíčové slovo.

Třída, která implementuje rozhraní, musí implementovat **všechny** metody rozhraní.

Příklad

```
<?php
interface Animal {
    public function makeSound();
}
```

```
class Cat implements Animal {
    public function makeSound() {
        echo "Meow";
    }
}
```

```
$animal = new Cat();
$animal->makeSound();
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_interface

výše uvedeného příkladu řekněme, že bychom chtěli napsat software, který spravuje skupinu zvířat. Existují akce, které mohou dělat všechna zvířata, ale každé zvíře to dělá svým vlastním způsobem.

Pomocí rozhraní můžeme napsat nějaký kód, který může fungovat pro všechna zvířata, i když se každé zvíře chová jinak:

Příklad

```
<?php
// Interface definition
interface Animal {
    public function makeSound();
}

// Class definitions
class Cat implements Animal {
    public function makeSound() {
        echo " Meow ";
    }
}

class Dog implements Animal {
    public function makeSound() {
        echo " Bark ";
    }
}

class Mouse implements Animal {
    public function makeSound() {
        echo " Squeak ";
    }
}

// Create a list of animals
$cat = new Cat();
$dog = new Dog();
$mouse = new Mouse();
$animals = array($cat, $dog, $mouse);

// Tell the animals to make a sound
foreach($animals as $animal) {
    $animal->makeSound();
}
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_interface2

Příklad vysvětlen

Cat, Dog a Mouse jsou všechny třídy, které implementují rozhraní Animal, což znamená, že všechny jsou schopny pomocí této `makeSound()` metody vydávat zvuk. Díky tomu můžeme procházet všemi zvířaty a říkat jim, aby vydali zvuk, i když nevíme, jaký druh zvířete každé z nich je.

Protože rozhraní neříká třídám, jak metodu implementovat, každé zvíře může vydávat zvuk svým vlastním způsobem.

PHP – co jsou vlastnosti?

PHP podporuje pouze jedinou dědičnost: podřízená třída může dědit pouze od jednoho jediného rodiče.

Co když tedy třída potřebuje zdědit více druhů chování? Tento problém řeší vlastnosti OOP.

Vlastnosti se používají k deklaraci metod, které lze použít ve více třídách. Vlastnosti mohou mít metody a abstraktní metody, které lze použít ve více třídách, a metody mohou mít jakýkoli modifikátor přístupu (veřejný, soukromý nebo chráněný).

Vlastnosti jsou deklarovány pomocí `trait` klíčového slova:

Syntax

```
<?php
trait TraitName {
    // some code...
}
?>
```

Chcete-li použít vlastnost ve třídě, použijte `use` klíčové slovo:

Syntax

```
<?php
class MyClass {
    use TraitName;
}
?>
```

Podívejme se na příklad:

Příklad

```
<?php
trait message1 {
    public function msg1() {
        echo "OOP is fun! ";
    }
}
```

```
class Welcome {
    use message1;
}
```

```
$obj = new Welcome();
$obj->msg1();
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_trait

Příklad vysvětlen

Zde deklarujeme jednu vlastnost: message1. Poté vytvoříme třídu: Vítejte. Třída používá vlastnost a všechny metody ve vlastnosti budou dostupné ve třídě.

Pokud jiné třídy potřebují používat funkci msg1(), jednoduše použijte v těchto třídách vlastnost message1. To snižuje duplicitu kódu, protože není potřeba znovu a znovu deklarovat stejnou metodu.

PHP - Použití více vlastností

Podívejme se na další příklad:

Příklad

```
<?php
trait message1 {
    public function msg1() {
        echo "OOP is fun! ";
    }
}
```

```
trait message2 {  
    public function msg2() {  
        echo "OOP reduces code duplication!";  
    }  
}
```

```
class Welcome {  
    use message1;  
}
```

```
class Welcome2 {  
    use message1, message2;  
}
```

```
$obj = new Welcome();  
$obj->msg1();  
echo "<br>";
```

```
$obj2 = new Welcome2();  
$obj2->msg1();  
$obj2->msg2();  
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_trait2

Příklad vysvětlen

Zde deklarujeme dvě vlastnosti: zpráva1 a zpráva2. Poté vytvoříme dvě třídy: Welcome a Welcome2. První třída (Welcome) používá vlastnost message1 a druhá třída (Welcome2) používá vlastnosti message1 i message2 (více vlastností je odděleno čárkou).

PHP - statické metody

Statické metody lze volat přímo – bez předchozího vytvoření instance třídy.

Statické metody jsou deklarovány pomocí **static** klíčového slova:

Syntax

```
<?php
class ClassName {
    public static function staticMethod() {
        echo "Hello World!";
    }
}
?>
```

Pro přístup ke statické metodě použijte název třídy, dvojtečku (::) a název metody:

Syntax

```
ClassName::staticMethod();
```

Podívejme se na příklad:

Příklad

```
<?php
class greeting {
    public static function welcome() {
        echo "Hello World!";
    }
}
```

```
// Call static method
greeting::welcome();
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_static_method

Zde deklarujeme statickou metodu: welcome(). Poté zavoláme statickou metodu pomocí názvu třídy, dvojtečky (::) a názvu metody (aniž bychom nejprve vytvořili instanci třídy).

HP – více o statických metodách

Třída může mít statické i nestatické metody. Ke statické metodě lze přistupovat z metody ve stejné třídě pomocí **self** klíčového slova a dvojtečky (::):

Příklad

```
<?php
class greeting {
    public static function welcome() {
        echo "Hello World!";
    }

    public function __construct() {
        self::welcome();
    }
}

new greeting();
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_static_method2

tatické metody lze také volat z metod v jiných třídách. Chcete-li to provést, statická metoda by měla být **public**:

Příklad

```
<?php
class greeting {
    public static function welcome() {
        echo "Hello World!";
    }
}

class SomeOtherClass {
    public function message() {
        greeting::welcome();
    }
}
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_static_method3

Chcete-li volat statickou metodu z podřazené třídy, použijte **parent** klíčové slovo uvnitř podřazené třídy. Zde může být statická metoda **public** nebo **protected**.

Příklad

```
<?php
class domain {
    protected static function getWebsiteName() {
        return "W3Schools.com";
    }
}

class domainW3 extends domain {
    public $websiteName;
    public function __construct() {
        $this->websiteName = parent::getWebsiteName();
    }
}

$domainW3 = new domainW3;
echo $domainW3->websiteName;
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_static_method4

PHP - Statické vlastnosti

Statické vlastnosti lze volat přímo – bez vytváření instance třídy.

Statické vlastnosti jsou deklarovány pomocí **static** klíčového slova:

Syntax

```
<?php
class ClassName {
    public static $staticProp = "W3Schools";
}
?>
```

Pro přístup ke statické vlastnosti použijte název třídy, dvojtečku (::) a název vlastnosti:

Syntax

```
ClassName::$staticProp;
```

Podívejme se na příklad:

Příklad

```
<?php
class pi {
    public static $value = 3.14159;
}
```

```
// Get static property
echo pi::$value;
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_static_prop

Příklad vysvětlen

Zde deklarujeme statickou vlastnost: \$value. Poté zopakujeme hodnotu statické vlastnosti pomocí názvu třídy, dvojtečky (::) a názvu vlastnosti (aniž bychom nejprve vytvořili třídu).

PHP – více o statických vlastnostech

Třída může mít statické i nestatické vlastnosti. Ke statické vlastnosti lze přistupovat z metody ve stejné třídě pomocí **self** klíčového slova a dvojtečky (::):

Příklad

```
<?php
class pi {
    public static $value=3.14159;
    public function staticValue() {
        return self::$value;
    }
}
```

```
$pi = new pi();
echo $pi->staticValue();
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_static_prop2

Chcete-li volat statickou vlastnost z podřízené třídy, použijte **parent** klíčové slovo uvnitř podřízené třídy:

Příklad

```
<?php
class pi {
    public static $value=3.14159;
}

class x extends pi {
    public function xStatic() {
        return parent::$value;
    }
}

// Get value of static property directly via child class
echo x::$value;

// or get value of static property via xStatic() method
$x = new x();
echo $x->xStatic();
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_static_prop3

!Jmenné prostory PHP

Jmenné prostory jsou kvalifikátory, které řeší dva různé problémy:

1. Umožňují lepší organizaci seskupením tříd, které spolupracují na plnění úkolu
2. Umožňují použití stejného názvu pro více než jednu třídu

Můžete například mít sadu tříd, které popisují tabulku HTML, jako je tabulka, řádek a buňka, a zároveň mít další sadu tříd pro popis nábytku, jako je stůl, židle a postel. Jmenné prostory lze použít k uspořádání tříd do dvou různých skupin a zároveň zabránit záměně dvou tříd Table a Table.

Deklarace jmenného prostoru

Jmenné prostory jsou deklarovány na začátku souboru pomocí **namespace** klíčového slova:

Syntax

Deklarujte jmenný prostor s názvem **Html**:

```
<?php
namespace Html;
?>
```

Poznámka: Deklarace **namespace** musí být první věcí v souboru PHP. Následující kód by byl neplatný:

```
<?php
echo "Hello World!";
namespace Html;
...
?>
```

Konstanty, třídy a funkce deklarované v tomto souboru budou patřit do jmenného prostoru **Html** :

Příklad

Vytvořte třídu **Table** v oboru názvů **Html**:

```
<?php
namespace Html;
class Table {
    public $title = "";
    public $numRows = 0;
    public function message() {
        echo "<p>Table '{$this->title}' has {$this->numRows} rows.</p>";
    }
}
$table = new Table();
$table->title = "My table";
$table->numRows = 5;
?>

<!DOCTYPE html>
<html>
```

```
<body>
```

```
<?php
```

```
$table->message();
```

```
?>
```

```
</body>
```

```
</html>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_namespace1

Pro další organizaci je možné mít vnořené jmenné prostory:

Syntax

Deklarujte jmenný prostor s názvem Html uvnitř jmenného prostoru s názvem Kód:

```
<?php
```

```
namespace Code\Html;
```

```
?>
```

Použití jmenných prostorů

Jakýkoli kód, který následuje po **namespace** deklaraci, funguje uvnitř jmenného prostoru, takže třídy, které patří do jmenného prostoru, lze konkretizovat bez jakýchkoli kvalifikátorů. Chcete-li přistupovat ke třídám z místa mimo jmenný prostor, třída musí mít jmenný prostor připojen k ní.

Příklad

Použijte třídy z oboru názvů Html:

```
<?php
```

```
$table = new Html\Table()
```

```
$row = new Html\Row();
```

```
?>
```

https://www.w3schools.com/php/showphp_classes.asp?filename=demo_classesphp

Když se současně používá mnoho tříd ze stejného jmenného prostoru, je jednodušší použít **namespace** klíčové slovo:

Příklad

Použijte třídy z oboru názvů Html bez potřeby Html\qualifier:

```
<?php
```

```
namespace Html;
```

```
$table = new Table();
```

```
$row = new Row();
```

```
?>
```

https://www.w3schools.com/php/showphp_classes.asp?filename=demo_classesphp_qual

!Alias jmenného prostoru

Může být užitečné dát jmennému prostoru nebo třídě alias, aby se usnadnilo psaní. To se provádí pomocí **use** klíčového slova:

Příklad

Dejte jmennému prostoru alias:

```
<?php
```

```
use Html as H;
```

```
$table = new H\Table();
```

```
?>
```

https://www.w3schools.com/php/showphp_classes.asp?filename=demo_classesphp_alias

Příklad

Dejte třídě alias:

```
<?php
```

```
use Html\Table as T;
```

```
$table = new T();
```

```
?>
```

https://www.w3schools.com/php/showphp_classes.asp?filename=demo_classesphp_alias_class

PHP – co je iterovatelný?

Iterovatelná je jakákoli hodnota, kterou lze procházet `foreach()` smyčkou.

Pseudotyp `iterable` byl zaveden v PHP 7.1 a lze jej použít jako datový typ pro argumenty funkcí a návratové hodnoty funkcí.

PHP - Použití Iterables

Klíčové `iterable` slovo lze použít jako datový typ argumentu funkce nebo jako návratový typ funkce:

Příklad

Použijte argument iterovatelné funkce:

```
<?php
function printIterable(iterable $myIterable) {
    foreach($myIterable as $item) {
        echo $item;
    }
}
```

```
$arr = ["a", "b", "c"];
printIterable($arr);
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_iterables

Příklad

Vrátit iterovatelný:

```
<?php
function getIterable():iterable {
    return ["a", "b", "c"];
}
```

```
$myIterable = getIterable();
foreach($myIterable as $item) {
    echo $item;
}
```



```
}  
?>
```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_iterables2

PHP – vytváření iterables

Pole

Všechna pole jsou iterovatelná, takže jakékoli pole lze použít jako argument funkce, která vyžaduje iterovatelnost.

Iterátory

Jakýkoli objekt, který implementuje **Iterator** rozhraní, lze použít jako argument funkce, která vyžaduje iterovatelnost.

Iterátor obsahuje seznam položek a poskytuje metody pro jejich procházení. Udržuje ukazatel na jeden z prvků v seznamu. Každá položka v seznamu by měla mít klíč, který lze použít k nalezení položky.

Iterátor musí mít tyto metody:

- **current()** - Vrátí prvek, na který aktuálně ukazuje ukazatel. Může to být jakýkoli datový typ
- **key()** Vrátí klíč spojený s aktuálním prvkem v seznamu. Může to být pouze celé číslo, float, boolean nebo řetězec
- **next()** Přesune ukazatel na další prvek v seznamu
- **rewind()** Přesune ukazatel na první prvek v seznamu
- **valid()** Pokud interní ukazatel neukazuje na žádný prvek (například pokud byla na konci seznamu volána funkce next(), mělo by to vrátit hodnotu false. V každém jiném případě vrátí hodnotu true

Příklad

Implementujte rozhraní Iterator a použijte jej jako iterovatelné:

```
<?php  
// Create an Iterator  
class MyIterator implements Iterator {  
    private $items = [];  
    private $pointer = 0;  
  
    public function __construct($items) {
```

```

    // array_values() makes sure that the keys are numbers
    $this->items = array_values($items);
}

public function current() {
    return $this->items[$this->pointer];
}

public function key() {
    return $this->pointer;
}

public function next() {
    $this->pointer++;
}

public function rewind() {
    $this->pointer = 0;
}

public function valid() {
    // count() indicates how many items are in the list
    return $this->pointer < count($this->items);
}
}

// A function that uses iterables
function printIterable(iterable $myIterable) {
    foreach($myIterable as $item) {
        echo $item;
    }
}

// Use the iterator as an iterable
$iterator = new MyIterator(["a", "b", "c"]);
printIterable($iterator);
?>

```

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_iterables_interface